



LABORATÓRIO DE IA
SYSTEM_READY

PI_IV::V7.0
CIÊNCIA DA COMPUTAÇÃO

● LOCAL-FIRST AI

RUST CORE

MLX.PILOT

Plataforma de **Execução e Automação**
de Modelos de Linguagem (LLMs).

STATUS DO PROJETO

Em Desenvolvimento

TECNOLOGIA

Rust + Tauri + Metal

EQUIPE DE DESENVOLVIMENTO

- > Kaike Vitorino
- > Livia Lemos
- > Marcello Augusto
- > Gabriel Lima
- > Gabriel Salustiano
- > Ramon Rocha



POWERED BY OPEN SOURCE

O Problema da IA Centralizada

Desafios técnicos e econômicos que motivaram o desenvolvimento da arquitetura local-first.



ERR_DEPENDENCY

Dependência Excessiva

Acesso a LLMs restrito a APIs proprietárias (OpenAI, Anthropic), criando "vendor lock-in" e vulnerabilidade a instabilidades de serviço externo.



ERR_HIGH_COST

Custos Elevados

Modelos de cobrança por token tornam inviável a experimentação acadêmica em larga escala e o uso contínuo em aplicações intensivas.



ERR_PRIVACY

Privacidade e Controle

Envio obrigatório de dados sensíveis para servidores de terceiros (Black Box), impossibilitando o uso em contextos de sigilo ou dados médicos/pessoais.



ERR_NO_HARDWARE

Experimentação Limitada

Dificuldade em testar arquiteturas customizadas, quantizações específicas (GGUF/AWQ) e otimizações de hardware local sem ferramentas adequadas.

A PROPOSTA MLX-PILOT

Uma plataforma desktop **Local-First** projetada para devolver o controle, reduzir custos e habilitar a pesquisa avançada em IA.



01

EXECUÇÃO LOCAL

Capacidade de rodar LLMs (Llama 3, Mistral, Gemma) diretamente no hardware do usuário, garantindo privacidade total dos dados e zero latência de rede.

STATUS: OFFLINE-READY



02

MULTI-BACKEND

Arquitetura agnóstica que unifica **MLX**, **llama.cpp** e **Ollama** sob uma única interface e API, permitindo flexibilidade total de infraestrutura.

STATUS: UNIFIED_API



03

EXPERIMENTAÇÃO

Sandbox controlado para estudantes e pesquisadores testarem hiperparâmetros, quantizações e arquiteturas de modelos sem custos de API.

STATUS: ACADEMIC_MODE



04

LABORATÓRIO PRÁTICO

Transforma qualquer computador com GPU/NPU em uma estação de trabalho de IA, servindo como base para disciplinas de Computação e TCCs.

STATUS: LAB_INSTALLED

Visão Arquitetural

CORE::RUST // UI::TAURI // PROVIDERS::MODULAR



Rust Core

Alta performance e memory safety



Daemon HTTP

Axum + Tokio (Async Runtime)



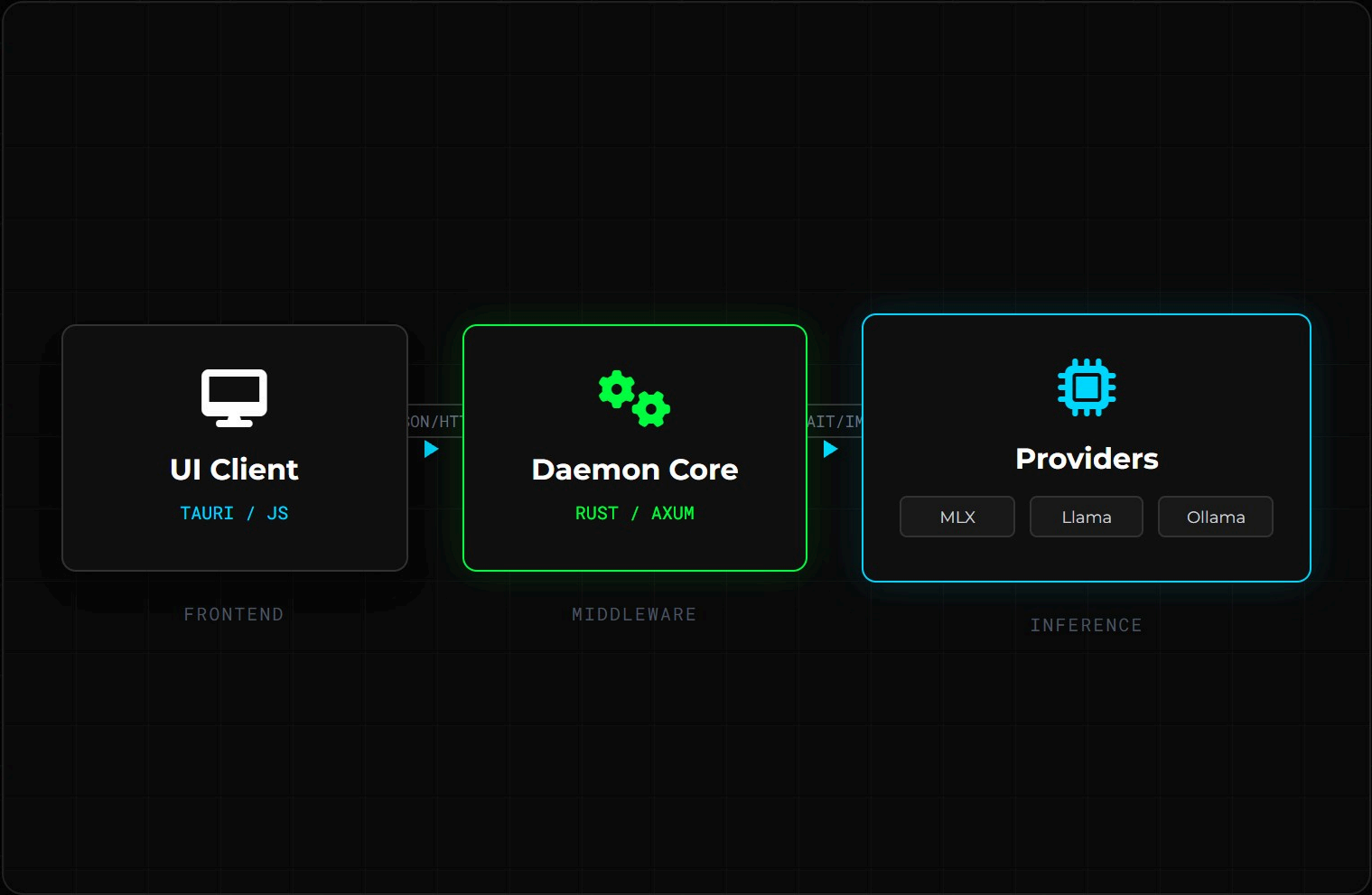
Sistema Modular

Arquitetura baseada em Providers



UI Desktop

Tauri (Webview nativa leve)



Fluxo de Execução

Toda requisição entra por um de três caminhos, cada um com sua rota no daemon, e todos terminam na mesma base compartilhada.

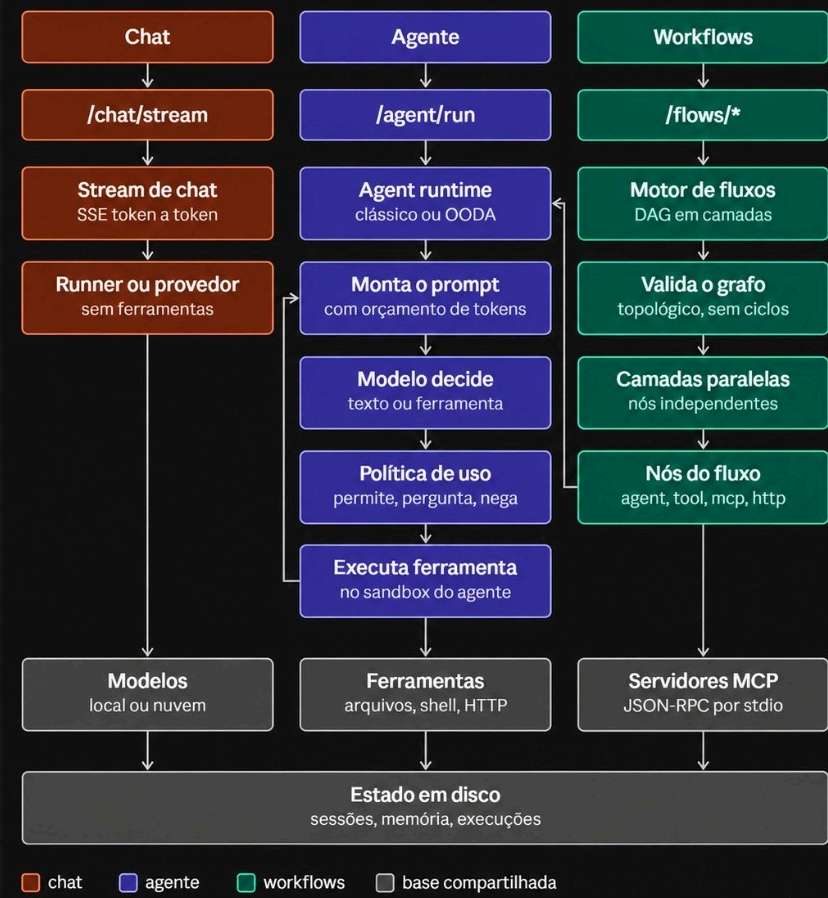
**Chat** `/chat/stream`
Resposta token a token via SSE, direto do runner ou provedor, sem ferramentas.

**Agente** `/agent/run`
Runtime clássico ou OODA: monta o prompt com orçamento de tokens, o modelo decide entre texto e ferramenta, a política permite, pergunta ou nega, e a ferramenta roda no sandbox, em ciclo.

**Workflows** `/flows/*`
Valida o grafo (DAG sem ciclos) e executa em paralelo as camadas de nós independentes: agent, tool, MCP ou HTTP. O nó agent reaproveita o runtime do agente.

**Base compartilhada**
Modelos locais ou em nuvem, ferramentas, servidores MCP e o estado em disco: sessões, memória e execuções.

MLX Pilot - fluxo do projeto



Backends Suportados

PROVIDER_SELECTION_MODE = "AUTO"

MLX

APPLE SILICON NATIVE



Framework de arrays otimizado para arquitetura Apple, permitindo acesso direto à memória unificada.

- METAL API
- UNIFIED MEMORY

- ✓ Otimização nativa para chips M1/M2/M3
- ✓ Lazy evaluation para alta performance
- ✓ Suporte a modelos Fine-tuned localmente

NPU UTILIZATIONHIGH

llama.cpp

CROSS-PLATFORM CORE



Motor de inferência C++ puro focado em máxima eficiência e compatibilidade de hardware.

- GGUF FORMAT
- CPU + GPU

- ✓ Quantização Avançada (4-bit, 8-bit)
- ✓ Suporte Híbrido (Offload parcial p/ GPU)
- ✓ Execução em hardware legado (AVX2)

COMPATIBILITYMAX

Ollama

SERVICE INTEGRATION



Camada de serviço que simplifica o gerenciamento de modelos e exposição via API.

- REST API
- MODEL MANAGER

- ✓ Download e gestão automática de Blobs
- ✓ Interface unificada para múltiplos modelos
- ✓ Fácil integração com ferramentas externas

EASE OF USEHIGH

Diferenciais Técnicos

SYSTEM_CAPABILITIES::V1.0 // INNOVATION_MODULE



Arquitetura Modular

Sistema plugável onde cada componente (UI, Core, Providers) opera de forma desacoplada, facilitando manutenção e upgrades.

```
trait Provider { fn infer(&self, prompt: str); }
```



Multi-Provider

Camada de abstração unificada que permite alternar entre backends (MLX, Llama, Ollama) em tempo de execução sem reiniciar.

```
let backend = ProviderFactory::create(config.type);
```



Streaming Real-time

Implementação eficiente de Server-Sent Events (SSE) para entrega de tokens com latência mínima na interface do usuário.

```
response.stream().map(|chunk|  
  Event::default().data(chunk))
```



Catálogo Hugging Face

Integração nativa com a API do Hugging Face para busca, filtragem e download automatizado de modelos GGUF e MLX diretamente pela interface.

```
GET https://huggingface.co/api/models?search=llama-3&filter=gguf
```

```
> searching models...  
> found: llama-3-8b-
```



Base para Agentes

Estrutura preparada para loop de feedback e tool-use, pavimentando o caminho para agentes autônomos locais.

```
agent.think().act().observe();
```

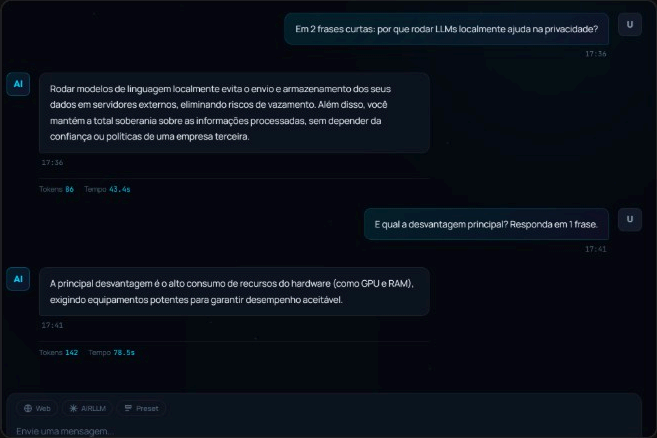
Demonstração Funcional

CURRENT_BUILD: v0.8.2-alpha

Chat com Streaming

REAL-TIME TOKEN GENERATION

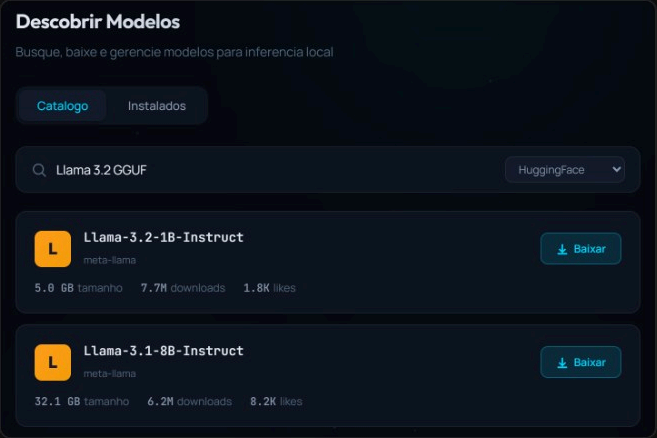
Interface de chat responsiva utilizando Server-Sent Events (SSE) para exibir tokens assim que são inferidos, sem latência perceptível.



Model Manager

HUGGING FACE INTEGRATION

Catálogo integrado para busca e download direto de modelos (GGUF/MLX) com verificação automática de hash e compatibilidade.



Seleção de Backend

HOT-SWAPPABLE PROVIDERS

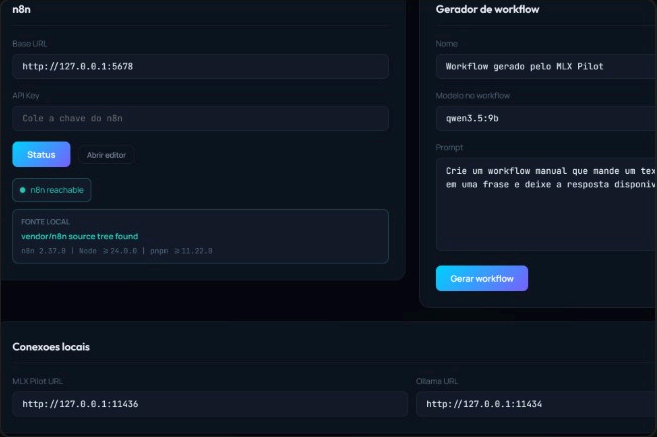
Capacidade de alternar dinamicamente entre motores de inferência (MLX, Llama.cpp, Ollama) sem reiniciar a aplicação.



Execução Local

PRIVACY & OFFLINE FIRST

Ambiente totalmente isolado que garante que nenhum dado do chat ou telemetria deixe a máquina local do usuário.



Simulação de Desempenho de Hardware

Adição de peças de CPU, RAM, GPU, VRAM e outros para simular como diferentes modelos de IA performam melhor em diferentes combinações de hardware.

Peças configuráveis

CPU, RAM, GPU, VRAM e demais componentes.

Combinações de hardware

Compara cenários montados a partir das peças.

Modelo x hardware

Mostra qual combinação favorece cada modelo de IA.

MLX Pilot - Hardware & Model Fit

Hardware & Model Fit

Detecte seu hardware e descubra quais modelos rodam bem

Modo: Simulado

Escanear Hardware

CPU

12th Gen Intel(R) Core(TM) i5-1235U

10 núcleos

RAM

32.0 GB

28.8 GB disponível

GPU

1 GPU - 24.0 GB VRAM

NVIDIA GeForce RTX 4090 (24.0 GB cuda)

BACKEND

CUDA

Aceleração GPU disponível

Simulador de Hardware Manual

GPU

NVIDIA GeForce RTX 4090 (24 GB)

QUANTIDADE

1

VRAM POR GPU (GB)

24

RAM DO SISTEMA

32 GB

BACKEND (OVERRIDE)

CUDA (NVIDIA)

Restaurar hardware real

Aplicar simulação

Modelos Ranqueados

Score

Geral

Modelo	Params	Quant	VRAM	Tok/s	Fit	Score	
Mixtral 8x7B Instruct	46.7B MoE	Q8_0	18.2 GB	102 tok/s	EXCELLENT	92	Baixar
Phi-3 Medium 128K Instruct	14.0B	Q8_0	17.3 GB	94 tok/s	EXCELLENT	89	Baixar
Llama 3 8B Instruct	8.0B	Q8_0	9.9 GB	164 tok/s	EXCELLENT	88	Baixar
DeepSeek R1 Distill Llama 8B	8.0B	Q8_0	9.9 GB	164 tok/s	EXCELLENT	88	Baixar
LLaVA 1.5 7B	7.0B	Q8_0	8.8 GB	187 tok/s	EXCELLENT	88	Baixar
Mistral 7B Instruct v0.3	7.3B	Q8_0	9.1 GB	180 tok/s	EXCELLENT	87	Baixar
Qwen 2.5 7B Instruct	7.6B	Q8_0	9.5 GB	172 tok/s	EXCELLENT	87	Baixar

RASTREABILIDADE

PR #42

Issue

d6951c4

1cce5ea

SYSTEM STATUS: DELIVERED

RUST TAURI METAL

Atualização de Documentação

Artefato que define e explicita os princípios, responsabilidades, controles e procedimentos utilizados pelo MLX Pilot para garantir o uso seguro, transparente, privado e auditável de modelos de inteligência artificial.

- 

Escopo de IA

Delimita o uso de modelos de IA no MLX Pilot.
- 

LGPD

Diretrizes de privacidade e proteção de dados pessoais.
- 

Classificações de risco

Categorização dos riscos do uso dos modelos.

Governança de IA do MLX Pilot

Última atualização: 2026-08-02

1. Objetivo

Este documento define os princípios, responsabilidades, controles e procedimentos utilizados pelo MLX Pilot para garantir o uso seguro, transparente, privado e auditável de modelos de inteligência artificial.

A governança descrita aqui se aplica ao produto real implementado neste repositório, incluindo chat com LLM, agente multi-turn, tool-calling, skills, memória/contexto, providers locais e remotos, escrita em arquivos, execução de comandos, conexões externas, cofre de segredos e logs de auditoria.

Este documento não substitui parecer jurídico. Ele estabelece regras técnicas e operacionais para orientar desenvolvimento, configuração, uso, revisão acadêmica e resposta a incidentes.

2. Escopo da IA

O MLX Pilot possui mais de uma superfície de IA. Cada superfície tem riscos e controles próprios.

Superfície	Onde aparece	Risco principal	Controles aplicáveis
Chat com LLM	/chat, /chat/stream, aba Chat	Resposta incorreta, envio de prompt a provider configurado	Transparência de provider/modelo, seleção local-first, aviso para provider remoto
Agente multi-turn	POST /agent/run	O modelo pode solicitar ferramentas em cadeia	AgentLoop, limite de iterações, PolicyEngine, ApprovalService, AuditLog
Tool-calling	read_file, write_file, edit_file, list_dir, exec, message, sessions_*, memory_*	Leitura/escrita indevida, envio de mensagens, execução de comando	Catálogo de tools, ToolRisk, profiles, sandbox, approvals e auditoria
Skills	SKILL.md no workspace	Ampliação de capacidades do agente	Capabilities declarativas, elegibilidade,

Providers locais	mlx, llamacpp, ollama	Consumo local de recursos, comportamento do modelo	Preferência local-first, limites de contexto, logs locais
Providers remotos	openai, anthropic, groq, openrouter, deepseek, custom	Envio de prompts/contexto a terceiros	Aviso explícito, API key via vault, minimização de contexto
Memória e contexto	MemoryStore, SessionStore, context budget	Retenção de informações sensíveis	Armazenamento local, exportação/deleção de sessões, controles de memória
Arquivos do workspace	Tools de filesystem	Vazamento, sobrescrita ou alteração accidental	Sandbox de path, paths sensíveis, owner-only mode, read-only/dry-run
Execução de comandos	exec	Comandos destrutivos ou escalada local	Denylist, safe bins, timeout, approval obrigatória por risco
Rede e serviços externos	Providers HTTP, Brave, channels, message	Egress não autorizado e envio de dados	Allowlist de domínios, airgapped mode, bloqueio de IP direto
Segredos	API keys, tokens, channel credentials, skill env	Exposição de credenciais	SecretsVault, api_key_ref, env_refs, redação em logs

3. Princípios

- Privacidade por padrão: o sistema deve processar localmente sempre que possível e enviar a terceiros apenas o necessário para a tarefa aprovada.
- Local-first: providers locais (mlx, llamacpp, ollama) devem ser preferidos quando atendem ao caso de uso.
- Transparência: a UI e as respostas operacionais devem indicar provider, modelo, modo de execução e quando dados podem sair da máquina.
- Supervisão humana: a IA não deve executar ações sensíveis sem aprovação ou política explícita.
- Menor privilégio: tools, skills, plugins e channels devem receber apenas as permissões necessárias.
- Rastreabilidade: prompts, chamadas de provider, tool calls, decisões de policy/approval e erros devem ser auditáveis.

Memória Local, Sessão Ativa e Contexto

Agora, as sessões são persistidas localmente para permitir a continuação de conversas e interações com modelos de IA sem perder contexto.

- 

Persistência local
Sessões gravadas na máquina do usuário.
- 

Sessão ativa
Retoma a conversa exatamente de onde parou.
- 

Contexto preservado
Interações com os modelos seguem sem perder contexto.

MLX Pilot - Histórico · sessão restaurada após reinício do daemon

Histórico de conversas

Organize, busque, ramifique (fork), edite e exporte suas sessões.

Buscar por nome...Mostrar arquivadasAtualizar

Nova conversa

qwen3.5-9b4 msgs03/10/2026, 18:08:14

FixarPastaForkArquivarExportar

Excluir

Create an n8n workflow JSON ob...

qwen3.5-9b2 msgs30/08/2026, 18:24:56

FixarPastaForkArquivarExportar

Excluir

Nova conversa

qwen3.5-9b0 msgs30/08/2026, 18:23:09

FixarPastaForkArquivarExportar

Excluir

Create an n8n workflow JSON ob...

qwen3.5-9b2 msgs30/08/2026, 18:05:29

FixarPastaForkArquivarExportar

Excluir

Create an n8n workflow JSON ob...

USER

Estamos preparando a apresentação do Projeto Integrador IV sobre o MLX Pilot. Guarde essa informação e responda apenas: Anotado.

EditarTruncar daqui

ASSISTANT

Anotado.

EditarTruncar daqui

USER

Sobre o que é a apresentação que estamos preparando? Responda em 1 frase.

EditarTruncar daqui

ASSISTANT

A apresentação é sobre o Projeto Integrador IV, focada no tema MLX Pilot.

EditarTruncar daqui

RASTREABILIDADE

 PR #15

 PR #31

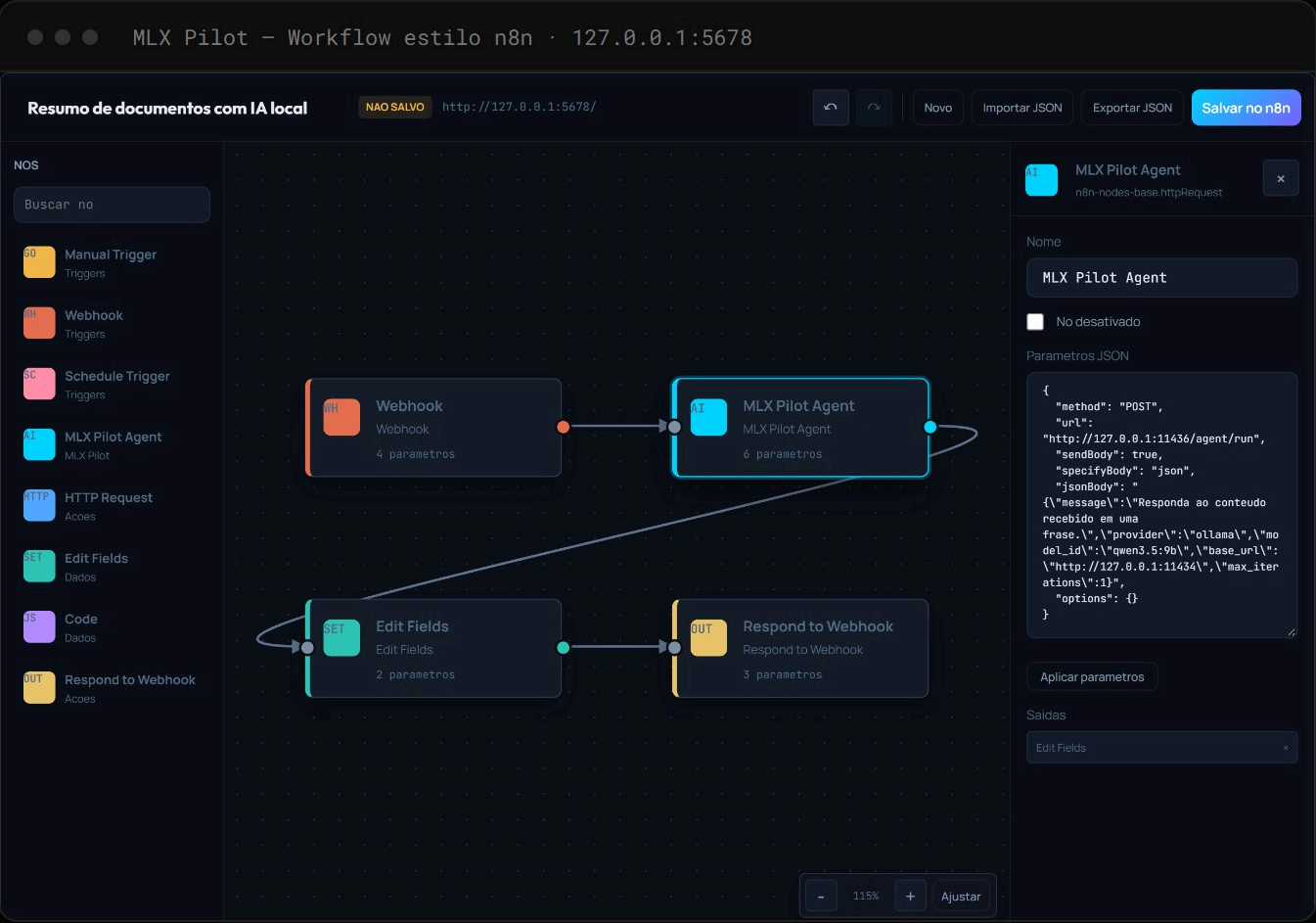
SYSTEM STATUS: DELIVERED

 RUST  TAURI  METAL

Integração com Workflow estilo n8n

O MLX-Pilot possui integração direta com um workflow estilo n8n local, podendo gerar workflows via prompt e um editor visual nativo para criar um workflow do zero.

-  **Workflow estilo n8n**
Integração direta com a instância local de workflow.
-  **Workflow via prompt**
Descreva a automação e o workflow é gerado.
-  **Editor visual nativo**
Crie um workflow do zero dentro do app.



Loop OODA

Implementação de ciclo OODA (Observe-Orient-Decide-Act) para tarefas multi-etapa. Dentro de um loop OODA, a IA pode desmembrar um objetivo complexo em várias sub-rotinas e percorrer todas elas de forma automatizada até terminar a tarefa.

- 

Tarefas multi-etapa

Objetivo complexo dividido em sub-rotinas.
- 

Ciclo automatizado

Observe, Orient, Decide e Act a cada sub-rotina.
- 

Até concluir

Percorre todas as sub-rotinas até o fim da tarefa.

```
terminal - POST /agent/run · runtime OODA · qwen3.5:9b local

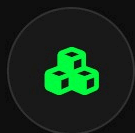
$ jq -c "{runtime_variant, model_id, message}" ooda_req.json
{"runtime_variant":"ooda","model_id":"qwen3.5:9b","message":"Descubra quantos arquivos .rs existem em src/, leia cada um e esc..."}
$ curl -s -X POST http://127.0.0.1:11435/agent/run --data @ooda_req.json \
  | jq '.ooda | del(.plan.steps[].result, .cycles[].outcome, .cycles[].step_description)'
{
  "stop_reason": "completed",
  "plan": {
    "objective": "Descubra quantos arquivos .rs existem em src/, leia cada um e escreva d..",
    "steps": [
      { "id": 1, "status": "done", "attempts": 1,
        "description": "Execute `find src/ -name \"*.rs\" | wc -l` to count .rs files in src/" },
      { "id": 2, "status": "done", "attempts": 1,
        "description": "Read each .rs file using `cat src/<filename>.rs` for every file found" },
      { "id": 3, "status": "done", "attempts": 1,
        "description": "Extract public function signatures (pub fn) from each file content" },
      { "id": 4, "status": "done", "attempts": 1,
        "description": "Create docs/analise.md if it does not exist" },
      { "id": 5, "status": "done", "attempts": 1,
        "description": "Write the function analysis to docs/analise.md with file names and their publ.." }
    ]
  },
  "cycles": [ // 5 ciclos
    { "cycle": 1, "decision": "execute", "step_id": 1, "tool_calls": 1, "elapsed_ms": 382610 },
    { "cycle": 2, "decision": "execute", "step_id": 2, "tool_calls": 4, "elapsed_ms": 1390706 },
    { "cycle": 3, "decision": "execute", "step_id": 3, "tool_calls": 4, "elapsed_ms": 504832 },
    { "cycle": 4, "decision": "execute", "step_id": 4, "tool_calls": 1, "elapsed_ms": 315842 },
    { "cycle": 5, "decision": "execute", "step_id": 5, "tool_calls": 5, "elapsed_ms": 2043103 },
  ],
  "tool_calls_made": 15, "replans": 0, "elapsed_ms": 5373892
}
$ cat docs/analise.md
# Análise de Funções Públicas

## src/config.rs
- `pub fn parse_config(path: &str) -> Config`
- `pub fn default_port() -> u16`

## src/main.rs
- Nenhuma função pública (apenas `fn main()` que não é pública)
```

Objetivo Acadêmico

RESEARCH_FOCUS // EXPERIMENTATION // BENCHMARK



01

ARQUITETURA MODULAR

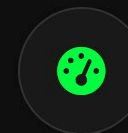
Estudo aprofundado sobre desacoplamento de sistemas, separando interface (Tauri) do controle lógico (Rust) e inferência.



02

LOCAL VS REMOTO

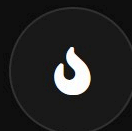
Análise comparativa crítica entre execução local (privacidade, latência zero) e nuvem (custo, escalabilidade).



03

BENCHMARK

Coleta sistemática de métricas de performance: Tokens por segundo (TPS), Tempo para o primeiro token (TTFT) e uso de memória.



04

STRESS TEST GPU

Avaliação de limites de hardware com modelos massivos (7B+), monitorando temperatura, consumo energético e gargalos de VRAM.



05

BASE DE PESQUISA

Criação de uma plataforma estável e documentada para servir como fundação para futuros Trabalhos de Conclusão de Curso (TCC).

Backlog Atual

FONTE: GITHUB ISSUES // MLX-PILOT/MLX-PILOT

• Aberta • Fechada

BACKLOG / FUTURO

2

S6#37

Sistema de plugins WASM para extensao

SEM SPRINT#40

Adição de métricas e insights por hardware personalizado

EM ANDAMENTO (S7-S8)

3

S7#34 · ABERTA

Hardening de seguranca (chat e runtime)

S8#25 · ABERTA

Canais oficiais e control plane

S8#32 · ABERTA

Release gate e smoke tests do MVP final

CONCLUÍDO

34

S5✓#36 · 01/10

~~Integracao de workflow estilo n8n~~

S7✓#31 · 26/09

~~Memoria local, sessao ativa e contexto~~

S6✓#35 · 14/09

~~Agente autonomo com loop OODA~~

+ 31 issues concluídas desde a S1

Expansão Acadêmica

PLANNING_PHASE



PRIORIDADE MÉDIA

Benchmark Comparativo

Execução de baterias de testes padronizados para medir TPS (Tokens Per Second) e latência entre diferentes backends (MLX vs Llama.cpp) e hardwares.

METRICS_COLLECTION →



PRIORIDADE MÉDIA

Stress Test GPU

Simulação de carga máxima em VRAM com modelos de parâmetros elevados (>13B) para validar estabilidade térmica e gerenciamento de memória.

HARDWARE_LIMITS →



PRIORIDADE MÉDIA

Observabilidade

Implementação de sistema robusto de logs estruturados e métricas em tempo real para monitorar o comportamento do daemon e inferências.

TELEMETRY_SYSTEM →



PRIORIDADE MÉDIA

Hardening de Segurança

Reforço das camadas de segurança, incluindo sanitização rigorosa de inputs no chat e isolamento (sandbox) do ambiente de execução de código.

SECURITY_AUDIT →

Evolução do Projeto

ROADMAP_TARGETS // v1.0.0-release

IN DEVELOPMENT



PHASE 1

Agente Autônomo

Loop OODA já implementado (PR #35). Próximo passo: ferramentas sandboxed e sistema de skills do agente.



PHASE 3

Sistema de Plugins

Arquitetura extensível baseada em WebAssembly (WASM) para permitir que a comunidade crie novos providers e ferramentas sem recompilar o core.



PHASE 2

Workflow estilo n8n

Workflow estilo n8n já integrado (PR #41). Próximo passo: embutir o motor no binário.



PHASE 4

Multiplataforma

Builds otimizadas para Windows (CUDA/DirectML) e Linux, com instaladores nativos (.msi, .deb) e gerenciamento de dependências isolado.



Tecnologias Utilizadas

HIGH_PERFORMANCE // MEMORY_SAFE // LOCAL_FIRST

CORE FOUNDATION



v1.75+

Rust

Linguagem principal garantindo segurança de memória, performance nativa e zero-cost abstractions.

Core Logic

Daemon



v2.0-beta

Tauri

Framework para construção de UI desktop leve e segura utilizando tecnologias web (HTML/JS/CSS).

Frontend

WebView



v0.7

Axum

Servidor HTTP ergonômico e modular para comunicação robusta entre UI e Daemon via API REST/SSE.

API REST

Handlers



Runtime

Tokio

Runtime assíncrono para gerenciar concorrência, I/O não bloqueante e tarefas em background.

Async

Concurrency

INFERENCE ENGINES



MLX

APPLE RESEARCH

Framework de array otimizado para Apple Silicon, permitindo inferência de alta performance em Macs (M1/M2/M3).

Metal API



llama.cpp

GGML / GGUF

Motor de inferência C++ puro com suporte a quantização (GGUF), rodando eficientemente em CPU e GPU variadas.

Cross-platform



Ollama

MODEL RUNNER

Interface de alto nível para gerenciamento simplificado de modelos locais e compatibilidade de API.

Easy Setup

Conclusão do Projeto

STATUS: READY_FOR_DEPLOYMENT // PI_IV_IN_PROGRESS

FUNCTIONAL SOFTWARE



Plataforma Técnica Real

O MLX-Pilot não é apenas um conceito teórico. É um software funcional capaz de baixar, carregar e executar inferência de LLMs localmente com performance mensurável.

MODULAR ARCHITECTURE



Projeto Escalável

A arquitetura baseada em *Providers* e desacoplamento entre UI (Tauri) e Core (Rust) permite a fácil adição de novos backends e funcionalidades sem refatoração massiva.

PRODUCTION READY CORE



Arquitetura Robusta

O uso de Rust garante segurança de memória (memory safety) e alta performance, criando uma fundação sólida para transformar o protótipo acadêmico em um produto.

ACADEMIC VALUE



Laboratório de IA

Potencial para se tornar a ferramenta padrão da instituição para ensino de IA Generativa, permitindo que alunos experimentem sem custos de nuvem ou configurações complexas.

Próximos Passos

SPRINT_STATUS: ACTIVE

Consolidação

O Projeto Integrador IV foca na estabilização da arquitetura, na evolução do agente e na coleta de métricas para validação acadêmica. O objetivo é entregar um artefato robusto e bem documentado.

PROJECT COMPLETION

85%

```
$ ./mlx-pilot --benchmark
Loading model weights... OK
Initializing Metal backend... OK
Running inference test [batch=1]...
Processing tokens...
```



Finalização do Benchmark

IN PROGRESS

Coleta sistemática de métricas de latência e consumo de VRAM em diferentes hardwares.



Implementação de Observabilidade

PENDING

Integração de dashboards para visualização de logs estruturados e tracing de requests.



Evolução do Agente

PLANNED

Ferramentas sandboxed, sistema de skills e API do agente integrada ao daemon, sobre o loop OODA já implementado.



Empacotamento e Entrega Final

PLANNED

Empacotamento, documentação técnica final e planejamento de entrega.